

Лабораторная работа по хранимым процедурам:

Хранимые процедуры (Stored Procedures) в PostgreSQL)

Цель работы

Научиться создавать и использовать хранимые процедуры в PostgreSQL для реализации бизнес-логики приложения на стороне сервера баз данных.

Общие требования

Предметная область: Выбрать выбранную предметную область: Для выбранной области необходимо создать таблицы и наполнить их тестовыми данными.

Оформление: Для каждой процедуры в отчете должны быть:

- Описание логики (что делает процедура).
- SQL-код создания процедуры.
- Пример вызова процедуры с параметрами.
- Ожидаемый результат выполнения

Лабораторная работа должна быть сделана в документе Word. Название файла должно быть в формате:

Номер_курса_название_группы_Фамилия_Имя_студента_номер_лабораторной_дата_сдачи.

Необходимо реализовать 12 хранимых процедур, сгруппированных по типам операций. Все процедуры должны принимать параметры. Они должны выполнять:

Операции чтения (SELECT)

1. Поиск сущности по идентификатору
2. Поиск сущностей по части названия/описания

```
CREATE TABLE products (  
    product_id SERIAL PRIMARY KEY,  
    product_name VARCHAR(150),  
    price NUMERIC(10, 2)  
);
```

```
CREATE OR REPLACE PROCEDURE find_products_by_name(p_name_part TEXT)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    -- Используем LIKE для поиска по части строки  
    SELECT * FROM products WHERE product_name ILIKE '%' || p_name_part || '%';  
END;  
$$;
```

```
-- Вызов процедуры  
CALL find_products_by_name('телефон');
```

5. Обновление одного поля по ID

6. Массовое обновление по условию

Усложненное задание (необязательно)

Повышение сложности позволяет отработать обработку ошибок, транзакции, динамический SQL и работу с курсорами.

7. Использование курсоров (Cursors) для постраничного вывода данных

Процедура не просто возвращает набор строк, а открывает курсор. Это необходимо, когда данных может быть очень много (тысячи записей), и возвращать их все сразу неэффективно. Процедура должна принимать параметры: номер страницы и размер страницы.

Примеры:

Процедура для вывода каталога книг по страницам, чтобы не перегружать клиентское приложение

```
CREATE OR REPLACE PROCEDURE получить_каталог_книг_по_страницам(
    p_номер_страницы INT,
    p_размер_страницы INT,
    OUT cur_books REFCURSOR
)
LANGUAGE plpgsql
AS $$
BEGIN
    -- Открываем курсор для выборки с ограничением и сдвигом
    OPEN cur_books FOR
        SELECT id_книги, название, автор, год_издания, доступно_экземпляров
        FROM книги
        ORDER BY название
        LIMIT p_размер_страницы
        OFFSET (p_номер_страницы - 1) * p_размер_страницы;
END;
$$;
```

Как вызывать:

```
BEGIN;
CALL получить_каталог_книг_по_страницам(1, 5);
FETCH ALL FROM "<unnamed portal 1>";
COMMIT;
```

∴

8. Динамический SQL для гибкого поиска книг

Универсальный поиск по названию или автору с возможностью сортировки по любой колонке.

```

CREATE OR REPLACE PROCEDURE поиск_книг(
    p_текст_поиска TEXT,
    p_колонка_сортировки VARCHAR,
    p_порядок_сортировки VARCHAR DEFAULT 'ASC'
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_sql TEXT;
BEGIN
    -- Проверка на допустимые значения для безопасности (защита от SQL-инъекций)
    IF p_колонка_сортировки NOT IN ('название', 'автор', 'год_издания') THEN
        RAISE EXCEPTION 'Недопустимая колонка для сортировки';
    END IF;

    IF p_порядок_сортировки NOT IN ('ASC', 'DESC') THEN
        RAISE EXCEPTION 'Недопустимый порядок сортировки';
    END IF;

    -- Формируем строку запроса
    v_sql := format(
        'SELECT * FROM книги WHERE название ILIKE %L OR автор ILIKE %L ORDER
        BY %I %s',
        '%' || p_текст_поиска || '%',
        '%' || p_текст_поиска || '%',
        p_колонка_сортировки,
        p_порядок_сортировки
    );

    -- Выполняем динамический запрос
    EXECUTE v_sql;
END;
$$;

```

Как вызывать:

```
CALL поиск_книг('Пушкин', 'год_издания', 'DESC');
```

9. Усложненные операции вставки (INSERT)

Вставка новой книги с обработкой конфликта (UPSERT). Логика: Если книга с таким ISBN уже есть в базе, не создавать новую запись, а увеличить количество экземпляров.

```
CREATE OR REPLACE PROCEDURE добавить_новую_книгу(  
    p_isbn VARCHAR,  
    p_название VARCHAR,  
    p_автор VARCHAR,  
    p_год_издания INT,  
    p_количество INT DEFAULT 1  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    INSERT INTO книги (isbn, название, автор, год_издания, количество_экземпляров,  
        доступно_экземпляров)  
    VALUES (p_isbn, p_название, p_автор, p_год_издания, p_количество, p_количество)  
    ON CONFLICT (isbn) DO UPDATE  
        SET количество_экземпляров = книги.количество_экземпляров +  
        EXCLUDED.количество_экземпляров,  
            доступно_экземпляров = книги.доступно_экземпляров +  
            EXCLUDED.количество_экземпляров;  
    RAISE NOTICE 'Операция выполнена для ISBN: %', p_isbn;  
END;  
$;
```

```
CALL добавить_новую_книгу('978-5-17-123456-7', 'Война и мир', 'Лев Толстой', 1869, 3);
```

```
CALL добавить_новую_книгу('978-5-17-123456-7', 'Война и мир', 'Лев Толстой', 1869, 2); -  
- Обновление (теперь 5 экз.)
```

Транзакционная регистрация нового читателя. Процедура регистрирует читателя. Если при вставке произойдет ошибка (например, дубликат email), транзакция откатывается.Примечание: В данном случае CALL работает в собственной транзакции по умолчанию.

```
CREATE OR REPLACE PROCEDURE зарегистрировать_читателя(  
    p_имя VARCHAR,  
    p_фамилия VARCHAR,  
    p_email VARCHAR  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    INSERT INTO читатели (имя, фамилия, email)  
    VALUES (p_имя, p_фамилия, p_email);  
  
    RAISE NOTICE 'Читатель % % успешно зарегистрирован', p_имя, p_фамилия;  
END;  
$$;
```